

Esercizio di programmazione concorrente

Ci sono tanti utenti, ciascuno implementato come un thread che, ciclicamente

- 1) chiede accesso alla risorsa;
- 2) quando ha ottenuto l'accesso alla risorsa, la usa per un po';
- 3) rilascia la risorsa;
- 4) fa altre operazioni che non richiedono l'uso della risorsa.

Mentre un utente usa la risorsa, gli altri utenti non la possono usare.

I metodi di accesso alla risorsa fanno in modo che gli utenti accedano alla risorsa nello stesso ordine con cui l'hanno richiesta. A questo scopo, la classe Risorsa implementa una coda in cui sono memorizzati gli identificatori degli utenti che hanno richiesto accesso alla risorsa.

Si richiede di modificare il codice dato in modo che il sistema si comporti come descritto sopra. Gli utenti che non possono accedere alla risorsa devono essere messi in attesa che si verifichino le condizioni per poter accedere (non è accettabile fare busy wait).

Il codice prodotto deve evitare deadlock, starvation, e altri problemi tipici della programmazione concorrente.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di Socket)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui la risorsa e gli utenti possono risiedere su macchine diverse. Il sistema deve essere realizzato usando socket per la comunicazione.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di RMI)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui la risorsa e gli utenti possono risiedere su macchine diverse. Il sistema deve essere realizzato usando RMI.

Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.